

## Laboratório 1 - Assembly RISC-V

### Grupo A8:

Gabriel de Medeiros Matos - 242001491

Gabriel Fonseca Pimenta - 242043428

Giovanni Daldegan - 232002520

Ricardo de Carvalho Nabuco - 231021360

Tauã Valentim de Albuquerque Martins Frade - 231021389

#### 1. Processador Uniciclo ISA RISC-V32I reduzida

##### 1.1. Análise do programa de1.s

Inicialização dos registradores e Memória:

→ O registrador gp recebe o valor **0x10010000**.

→ Na posição de memória 0(gp) está armazenado o valor **0xFFFFFFFF0F**.

Sequência de Instruções

##### 1. Carrega valor de memória em t1

- **lw t1, 0(gp) → t1 = 0xFFFFFFFF0F**

##### 2. Define valor imediato em t2

- **addi t2, zero, 0x777 → t2 = 0x00000777**

##### 3. AND bit a bit

- **and t0, t1, t2 → t0 = 0x00000007**

##### 4. OR bit a bit

- **or t0, t1, t2 → t0 = 0xFFFFFFFF7F**

##### 5. Soma

- **add t0, t2, t1 → t0 = 0x00000686 (considerando overflow)**

##### 6. Subtração

- **sub t0, t2, t1 → t0 = 0x00000868**

##### 7. Set Less Than (comparação com sinal)

- **slt t0, t1, t2 → t0 = 1** (t1 < t2, pois t1 é negativo e t2 positivo)
- **slt t0, t2, t1 → t0 = 0** (t2 não é menor que t1)

##### 8. Desvio condicional

- **beq t0, zero, PULA → Como t0 = 0, o fluxo salta para o label PULA.**

## Fluxo do Programa

- Após o salto condicional, o programa executa:
  - `jal PROC` → Chama o procedimento `PROC` e salva o endereço de retorno.
  - `addi t0, zero, 0xFFFFCCC` → `t0 = 0xFFFFCCC` (valor negativo).

## Procedimento PROC

- `li t0, 127` → `t0 = 127`
- `sw t0, 4(gp)` → Salva 127 na posição `4(gp)`
- `lw t0, 0(gp)` → Carrega valor original da memória (`0xFFFFF0F`)
- `lw t0, 4(gp)` → Recupera valor salvo (`127`)
- `ret` → Retorna ao ponto de chamada.

### 1.2. Banco de Registradores

O banco de registradores utilizado foi aproveitado do arcabouço fornecido (`TopDE.qar`). Na descrição de hardware em Verilog, ele possui um vetor de 32 registradores de 32 bits, além de permitir a leitura assíncrona de dois registradores (`rs1` e `rs2`).

Sua escrita é síncrona, ocorrendo apenas na subida de clock ou na subida do sinal de reset. Ao reiniciar o banco, o registrador correspondente ao stack pointer é redefinido e todos os demais são zerados.

### 1.3. Gerador de Imediatos

O Gerador de Imediatos foi feito em Verilog, com a entrada e saída de 32 bits. O gerador funciona de forma assíncrona. É implementado por meio de um ‘case’ que define o formato da saída de acordo com o tipo da instrução de entrada.

A identificação da entrada é auxiliada pela definição dos tipos no arquivo "Parametros.v", que funciona como um dicionário que associa valores binários a palavras chave, deixando o código mais legível. Assim, usando a estrutura das instruções no Assembly RISC-V 32 bits, os bits da entrada são arranjados para formar o imediato. Usando extensão de sinal em todos os casos, exceto nas instruções tipo U (`lui`) que são definidas por deslocamento à esquerda.

A saída padrão é 0.

## CORE INSTRUCTION FORMATS

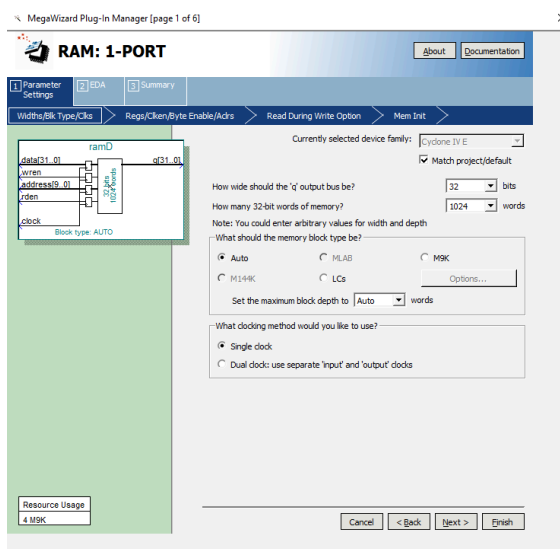
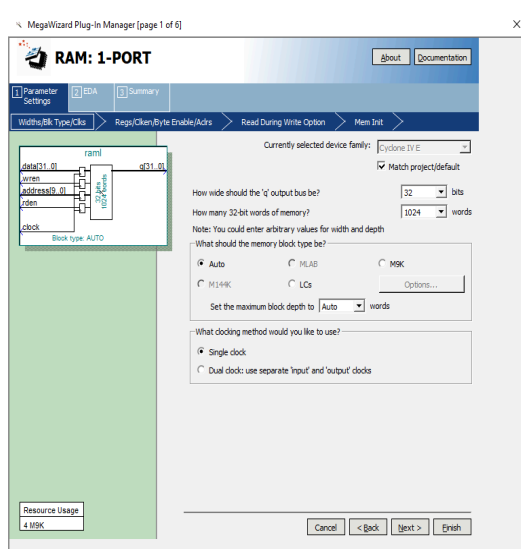
|          | 31                    | 25 24 | 20 19 | 15 14  | 12 11       | 7 6    | 0 |
|----------|-----------------------|-------|-------|--------|-------------|--------|---|
| <b>R</b> | funct7                | rs2   | rs1   | funct3 | Rd          | opcode |   |
| <b>I</b> | imm[11:0]             |       | rs1   | funct3 | Rd          | opcode |   |
| <b>S</b> | imm[11:5]             | rs2   | rs1   | funct3 | imm[4:0]    | opcode |   |
| <b>B</b> | imm[12 10:5]          | rs2   | rs1   | funct3 | imm[4:1 11] | opcode |   |
| <b>U</b> | imm[31:12]            |       |       |        | Rd          | opcode |   |
| <b>J</b> | imm[20 10:1 11 19:12] |       |       |        | Rd          | opcode |   |

### 1.4. Memória de dados e programa

Os blocos de memória de dados e memória de programa também foram aproveitados do arcabouço fornecido nos arquivos do laboratório. Para esse laboratório, as memórias correspondem aos dados e instruções do programa de1.s, exportadas pelo RARS em arquivos .mif que podem ser utilizados para futuras simulações.

Além disso, exportamos o conteúdo das memórias de instrução e dados do programa desenvolvido no Rars16\_Custom2 usando a opção “Dump Memory” no formato MIF 32. Os arquivos gerados (de1\_text.mif) e (de1\_data.mif) foram utilizados no Quartus para inicializar as memórias do processador, garantindo que as instruções e dados do simulador foram corretamente carregados na FPGA.

O processo foi finalizado com a compilação do projeto, validando a integração dos conteúdos da RAM e ROM conforme os requisitos de endereço e tamanho definidos pelo roteiro.



### 1.5. ULA reduzida

A ULA do processador foi aproveitada do arcabouço fornecido no Aprender3. A ULA é um componente combinacional que é implementada em verilog usando a definição “Parametros.v” em um ‘case’.

As entradas da ULA são iControl, de 5 bits, que define a operação da ULA. e duas entradas de 32 bits, iA e iB, que são os operandos. Há uma saída de 32 bits oResult, referente ao resultado da operação, e zero, de 1 bit, que indica se o resultado da operação é igual a 0.

As operações são implementadas diretamente com Verilog.

### 1.6. Blocos de Controle e Controle da ULA - giovanni

Os blocos de controle foram implementados pela descrição de circuitos combinacionais.

O Bloco de Controle principal recebe o opcode da instrução lida da memória de programa e define os sinais de controle:

- Mem2Reg: define a origem do dado para escrever no banco de registradores.
- LeMem: determina se deve haver leitura da memória de dados.
- EscreveMem: determina se deve haver escrita na memória de dados.
- Branch: sinaliza se a instrução é do tipo B, um branch.
- ALUOp: diferencia se a instrução lida é
- OrigULA: seleciona a saída do multiplexador para o segundo operador da ULA.
- EscreveReg: define se deve haver escrita no banco de registradores.

```

16 `ifndef PARAM
17 `include "Parametros.v"
18 `endif
19
20 module Controle (
21     input wire [6:0] opcode,
22     output wire      Mem2Reg, LeMem, EscreveMem, Branch, OrigULA, EscreveReg,
23     output wire [1:0] opULA
24 );
25
26
27 always @(*)
28 begin
29     case (opcode)
30     OPC_RTYPE: begin // 33 - add sub slt and or
31         EscreveReg <= 1'b1;
32         Mem2Reg    <= 1'b0;
33         LeMem      <= 1'b0;
34         EscreveMem <= 1'b0;
35         Branch     <= 1'b0;
36         OrigULA    <= 1'b0;
37         opULA      <= 2'b10;
38     end
39     OPC_OPIMM: begin // 13 - addi
40         EscreveReg <= 1'b1;
41         Mem2Reg    <= 1'b0;
42         LeMem      <= 1'b0;
43         EscreveMem <= 1'b0;
44         Branch     <= 1'b0;
45         OrigULA    <= 1'b1;
46         opULA      <= 2'b00;
47     end
48     OPC_LUI: begin // 37 - lui
49         EscreveReg <= 1'b1;
50         Mem2Reg    <= 1'b0;
51         LeMem      <= 1'b0;
52         EscreveMem <= 1'b0;

```

O bloco de Controle da ULA recebe ALUop como input e define os sinais que controlam qual operação será executada pela ULA.

```

1  /*
2  * Bloco de controle (combinacional) da ULA
3  *
4  * operações:
5  * 00 - add
6  * 01 - sub
7  * 10 - funct
8  */
9
10 module ControleULA (
11     input wire [1:0] opULA,
12     input wire [2:0] funct3,
13     input wire [6:0] funct7,
14     output wire [4:0] codULA
15 );
16
17 always @(*)
18 begin
19     case (opULA)
20     2'b00:
21         codULA <= OPADD; // addi lui lw sw beq jal jalr
22     2'b01:
23         codULA <= OPSUB; // beq
24     2'b10:
25         case (funct3) // tipo-R: add sub slt or and
26         3'b000:
27             begin
28                 if (funct7 == 7'b0)
29                     codULA <= OPADD;
30                 else
31                     codULA <= OPSUB;
32             end
33         3'b010:
34             codULA <= OPSLT;
35         3'b110:
36             codULA <= OPOR;
37         3'b111:
38             codULA <= OPAND;
39         endcase
40     endcase
41 end
42
43 endmodule
44

```

## 1.7. Processador Uniciclo

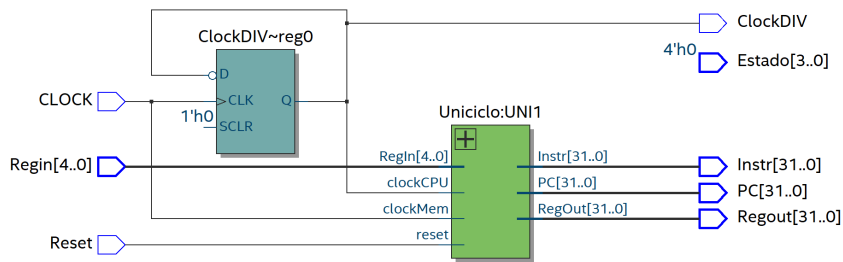
### a) RTL view

As visualizações RLT foram obtidas seguindo os seguintes passos: Tools -> Netlist Viwer -> RLT Viwer

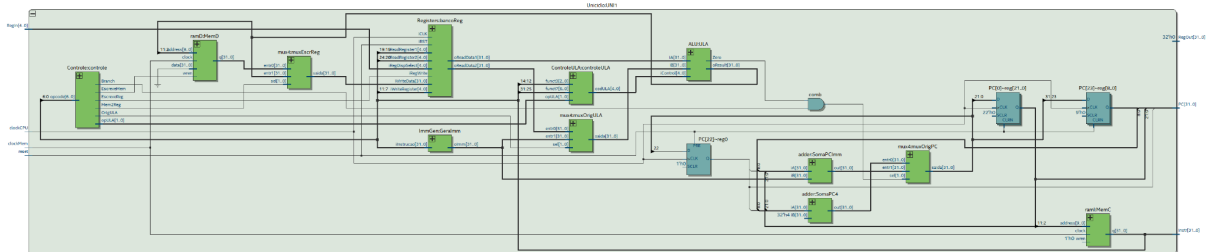
A primeira vizualização, TopDE, apresenta um componente “Uniciclo:UNI1”, em que ao clicar no “+”, é possível obter a vizualização do processador uniciclo e, indo em cada módulo do processador, é possível vizualizar o Banco de Registradores, o Controle, o Controle ULA, o Gerador de Imediatos e a ULA.

Abaixo, seguem as imagens das visualizações:

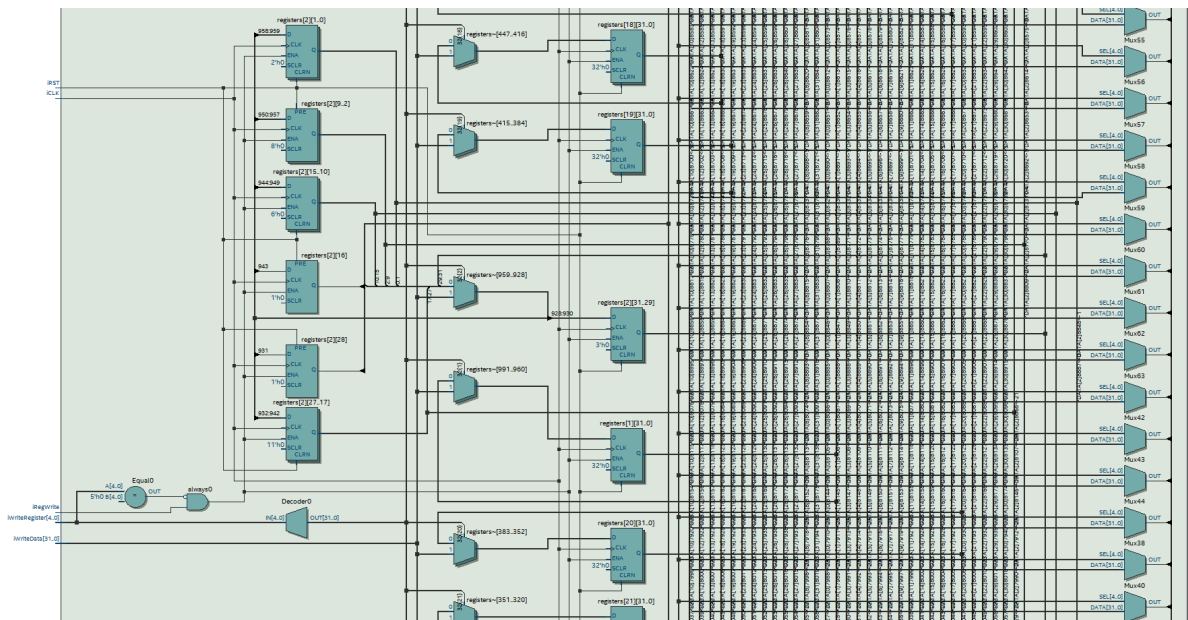
TopDE



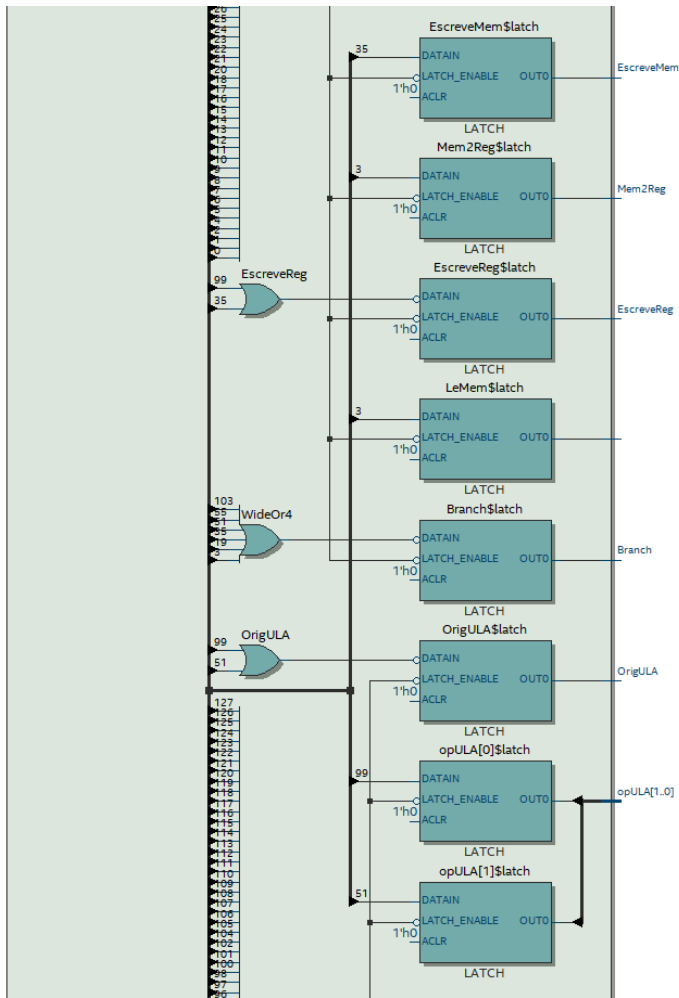
## Uniciclo Completo



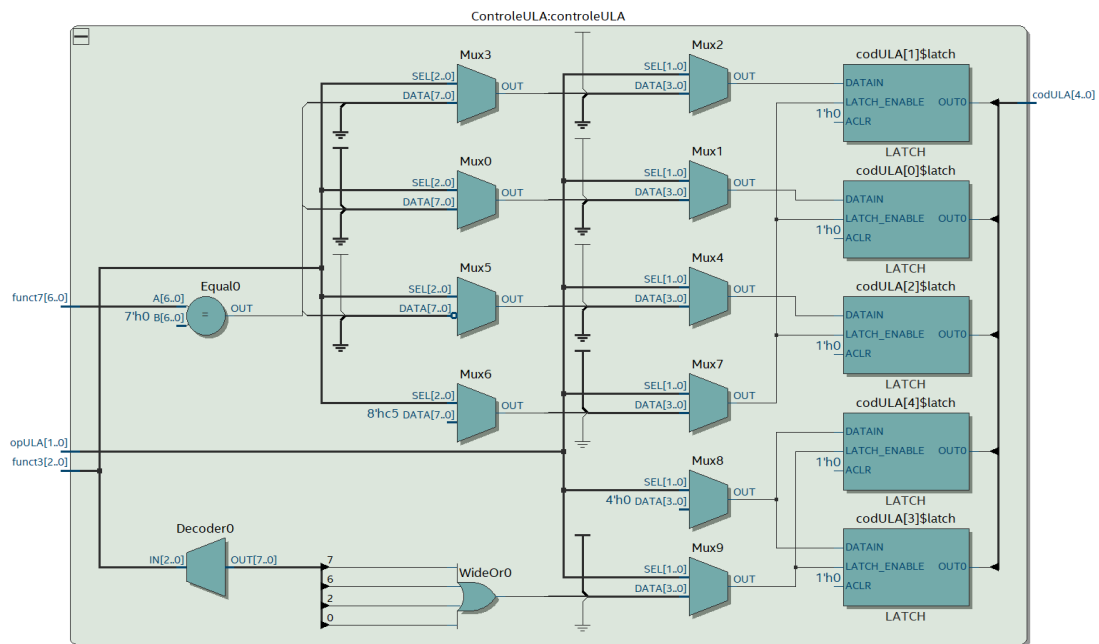
## Banco Registradores



## Controle

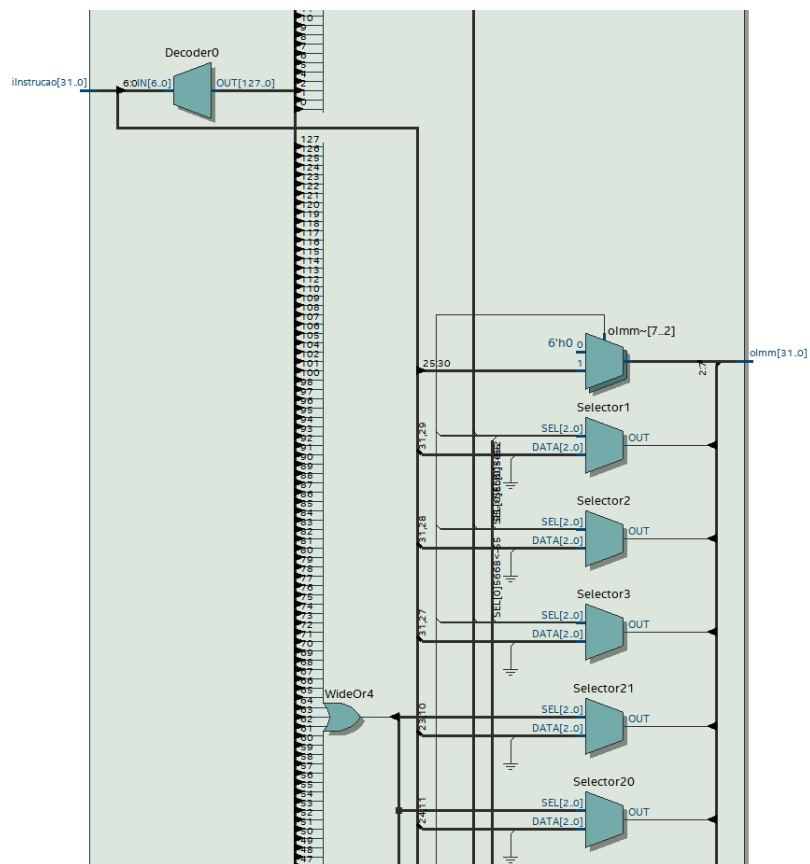


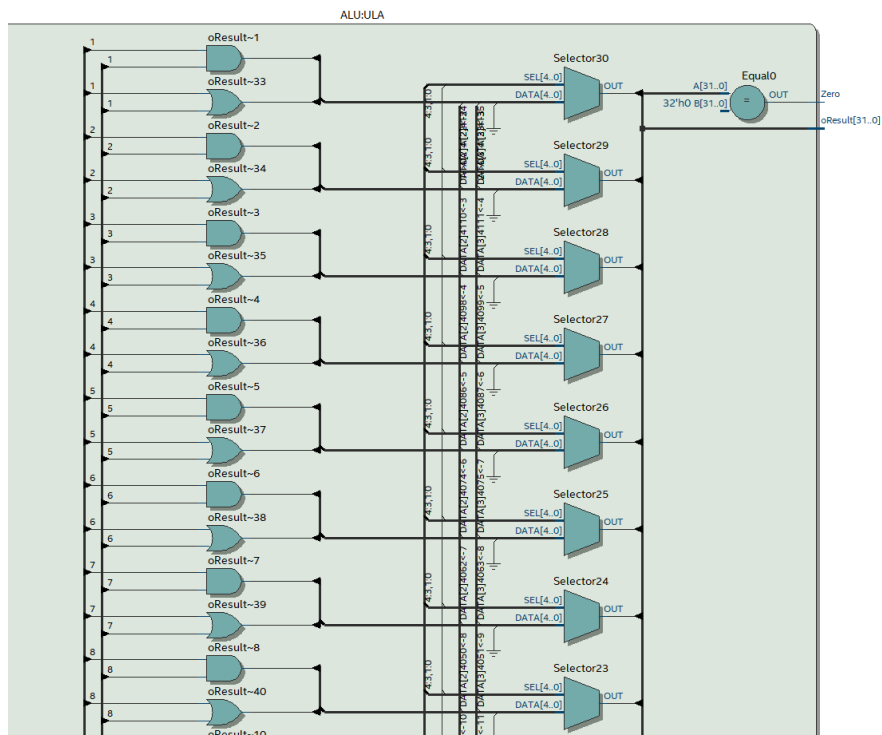
## Controle ULA





## ULA





## b) Requisitos físicos e funcionais

Os resultados físicos foram obtidos após acionar o botão de “Start Compilation”, em que foi aberto o painel de “Flow Summery”, com informações sobre o total de elementos lógicos, o total de registradores, o número de bits na memória, entre outros.

## Físicos

| Flow Summary                    |                                                 |
|---------------------------------|-------------------------------------------------|
| <<Filter>>                      |                                                 |
| Flow Status                     | Successful - Mon Nov 10 00:03:43 2025           |
| Quartus Prime Version           | 25.1std.0 Build 1129 10/21/2025 SC Lite Edition |
| Revision Name                   | TopDE                                           |
| Top-level Entity Name           | TopDE                                           |
| Family                          | Cyclone V                                       |
| Device                          | 5CGXFC7C7F23C8                                  |
| Timing Models                   | Final                                           |
| Logic utilization (in ALMs)     | 1,136 / 56,480 ( 2 % )                          |
| Total registers                 | 1296                                            |
| Total pins                      | 108 / 268 ( 40 % )                              |
| Total virtual pins              | 0                                               |
| Total block memory bits         | 65,536 / 7,024,640 ( < 1 % )                    |
| Total DSP Blocks                | 0 / 156 ( 0 % )                                 |
| Total HSSI RX PCSs              | 0 / 6 ( 0 % )                                   |
| Total HSSI PMA RX Deserializers | 0 / 6 ( 0 % )                                   |
| Total HSSI TX PCSs              | 0 / 6 ( 0 % )                                   |
| Total HSSI PMA TX Serializers   | 0 / 6 ( 0 % )                                   |
| Total PLLs                      | 0 / 13 ( 0 % )                                  |
| Total DLLs                      | 0 / 4 ( 0 % )                                   |

Os requisitos temporais foram obtidos por meio do “Time Analyser”, indo em “create timing netlist”, criando um clock de 20 ns, clicando em “read SDC file”, vendo as informações de clock na pasta “Slack”, em Report Setup Summary e Report Hold Summary

Abaixo, seguem os valores de clock, os quais, por serem positivos, indicam que não houve nenhum erro evidente no experimento:

Temporais

|   | Clock | Slack | End Point TNS |
|---|-------|-------|---------------|
| 1 | CLK1  | 0.136 | 0.000         |

|   | Clock | Slack | End Point TNS |
|---|-------|-------|---------------|
| 1 | CLK1  | 1.042 | 0.000         |

#### c) Simulação

Não foi possível realizar a simulação devido a um erro no momento de importar a memória de dados e de instruções, dos arquivos .mif, para dentro do quartus

#### d) Frequência máxima de clock

A frequência máxima foi obtida na parte do item b), indo em Datasheet -> Report Fmax

|   | Fmax      | Restricted Fmax | Clock Name | Note |
|---|-----------|-----------------|------------|------|
| 1 | 50.34 MHz | 50.34 MHz       | CLK1       |      |

## 2. Vídeo de apresentação

<https://youtu.be/sKh4Z78-Vhs>

## 3. Referências bibliográficas

1. RISC-V-Reference-Guide\_v24 - Core instruction formats